Low-Level Design (LLD)

Project Title: RAG-Based Customer Support Assistant using LangGraph and Human-in-the-Loop

---

1. Module-Level Design

1.1 Document Processing Module

- Input: PDF file
- Output: Raw extracted text
- Responsibility: Extract text using PDF parsers (PyPDF / LangChain loaders)

1.2 Chunking Module

- Input: Raw text
- Output: List of chunks
- Strategy:
    - Chunk size: 500–1000 tokens
    - Overlap: 100–200 tokens
- Purpose: Maintain context continuity

1.3 Embedding Module

- Input: Text chunks
- Output: Vector embeddings
- Model: OpenAI embeddings / Sentence Transformers

1.4 Vector Storage Module

- Tool: ChromaDB
- Stores:
    - Chunk ID
    - Text
    - Embedding vector
- Indexing: Cosine similarity

1.5 Retrieval Module

- Input: Query embedding
- Process: Top-K similarity search
- Output: Relevant chunks

1.6 Query Processing Module

- Input: User query
- Steps:
    - Intent detection
    - Query embedding
    - Retrieval
    - Response generation

1.7 Graph Execution Module (LangGraph)

- Controls workflow execution
- Handles node transitions and state updates

1.8 HITL Module

- Handles escalation
- Routes query to human
- Stores pending queries

---

2. Data Structures

2.1 Document Representation

{ “doc_id”: “string”, “text”: “string” }

2.2 Chunk Format

{ “chunk_id”: “string”, “doc_id”: “string”, “text”: “string”, “embedding”: [float] }

2.3 Embedding Structure

{ “vector”: [float] }

2.4 Query-Response Schema

{ “query”: “string”, “intent”: “FAQ | COMPLEX | ESCALATION”, “response”: “string”, “confidence”: “float”, “sources”: [“chunk_id”] }

2.5 Graph State Object

{ “query”: “string”, “intent”: “string”, “retrieved_chunks”: [], “response”: “string”, “confidence”: “float”, “escalation”: “boolean” }

---

3. Workflow Design (LangGraph)

Nodes

- Input Node
- Processing Node (Intent + Retrieval)
- LLM Node (Generation)
- Decision Node
- Output Node
- HITL Node

Edges

Input → Processing → LLM → Decision Decision → Output (if confident) Decision → HITL (if not confident)

State Flow

State is passed between nodes and updated at each stage:

- Query added at input
- Intent added at processing
- Retrieved chunks added before LLM
- Response + confidence added after LLM

---

4. Conditional Routing Logic

Answer Generation Criteria

- Relevant chunks retrieved
- Confidence above threshold (e.g., 0.7)

Escalation Criteria

- Low confidence (< threshold)
- No relevant chunks found
- Query classified as complex or sensitive

Conditions

- Low confidence → escalate
- Missing context → escalate
- Complex query → optional refinement or escalate

---

5. HITL Design

When Escalation is Triggered

- Confidence too low
- Retrieval fails
- Query flagged sensitive

After Escalation

- Query stored in queue
- Assigned to human agent

Integration of Human Response

- Human response stored
- Sent back to user
- Optionally logged for future learning

---

6. API / Interface Design

6.1 Upload PDF

POST /upload

Request: { “file”: “pdf” }

Response: { “status”: “uploaded” }

6.2 Query API

POST /query

Request: { “query”: “string” }

Response: { “response”: “string”, “confidence”: “float”, “sources”: [] }

Interaction Flow

User → API → LangGraph → Response

---

7. Error Handling

Missing Data

- Return validation error

No Relevant Chunks Found

- Return fallback message OR escalate

LLM Failure

- Retry mechanism
- If fails again → escalate to HITL

---

8. Conclusion

This Low-Level Design provides a detailed breakdown of modules, data structures, workflows, and routing logic required to implement a RAG-based customer support assistant. The design ensures scalability, reliability, and flexibility for real-world deployment.